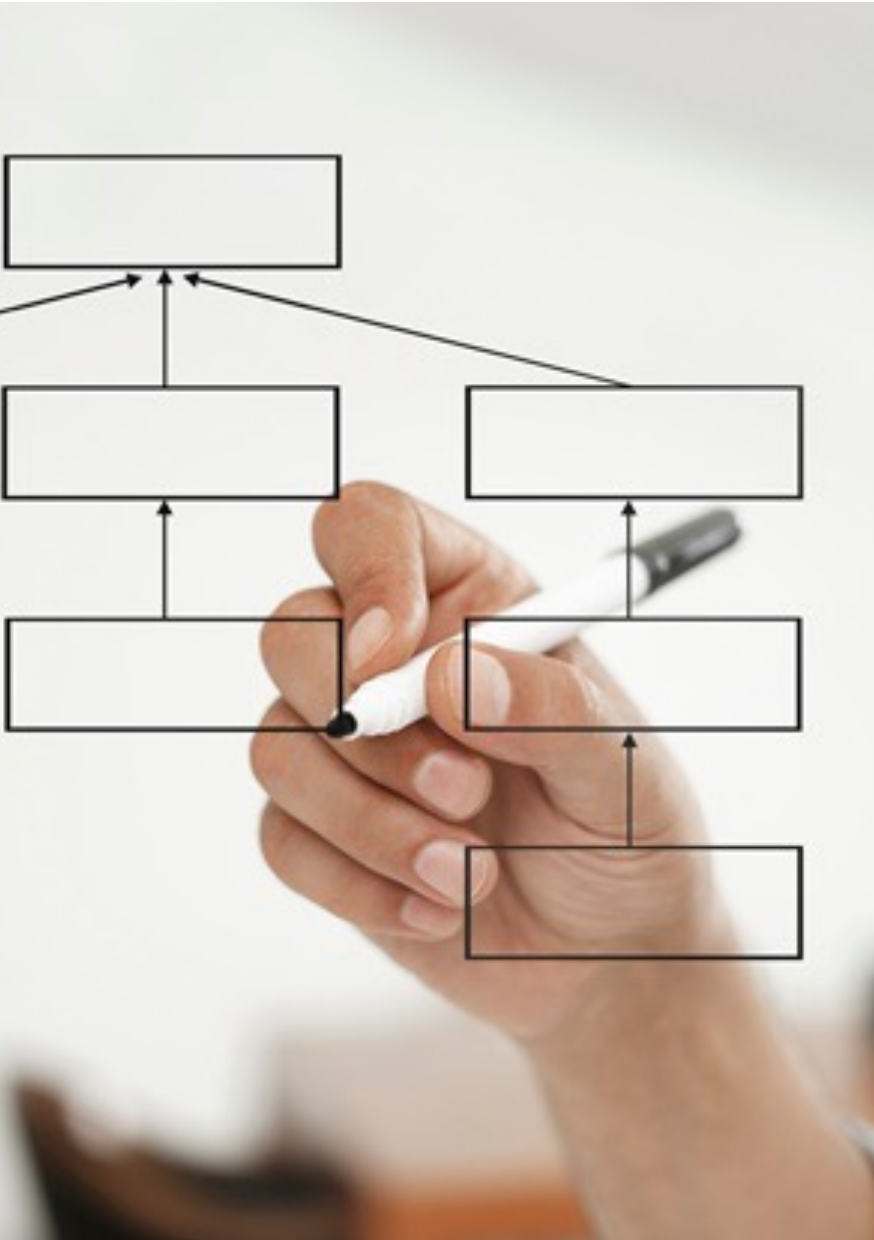
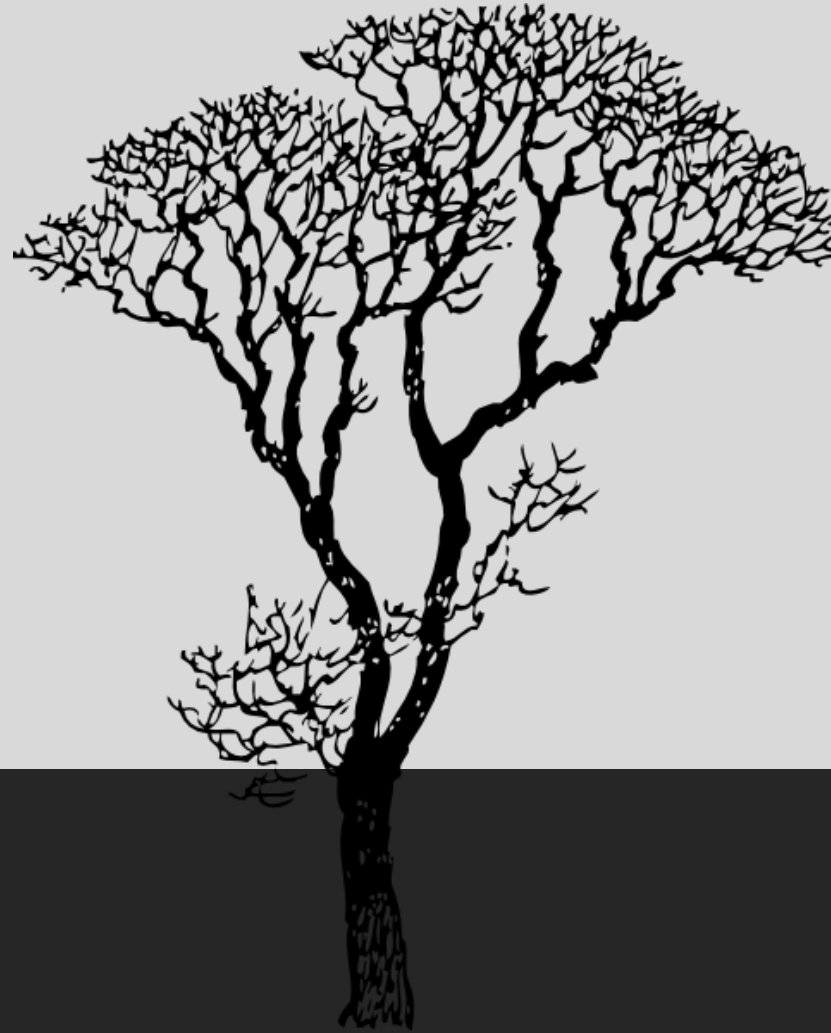




Estrutura de Dados I

CÁSSIO CAPUCHO PEÇANHA - 12

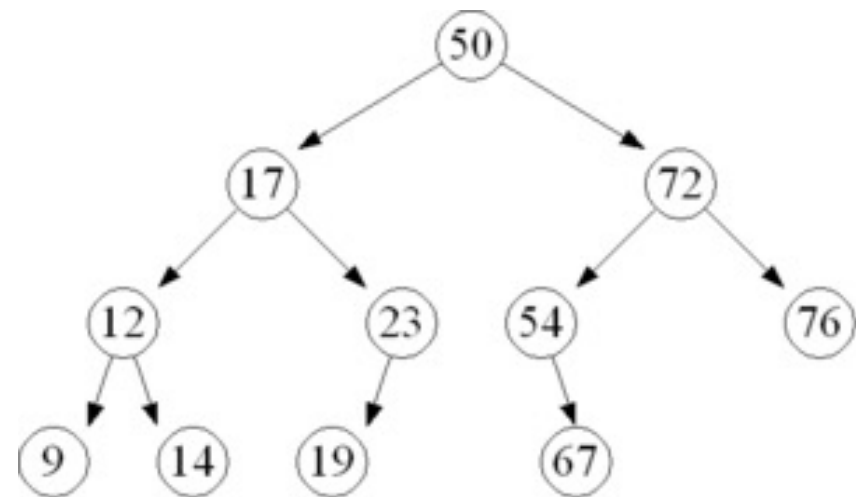
Árvore



05/05/2026

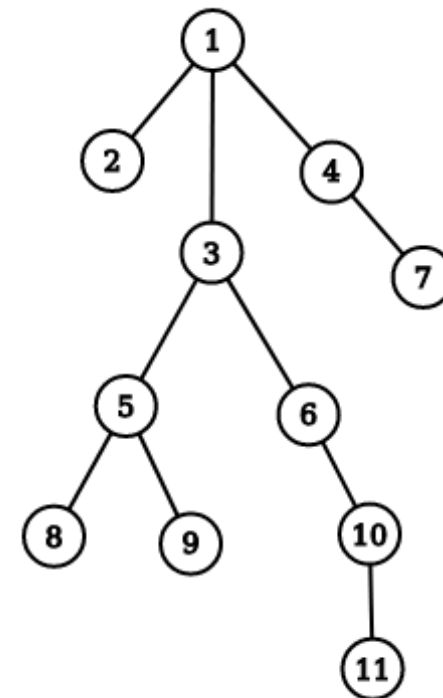
Árvores

- São um tipo especial de grafo;
- Qualquer par de vértices está conectado a apenas uma aresta;
- Grafo conexo
 - Existe exatamente um caminho entre quaisquer dois de seus vértices
- Acíclico
 - Não possui ciclos
- Adequadas para representar estruturas hierárquicas não lineares



Árvores

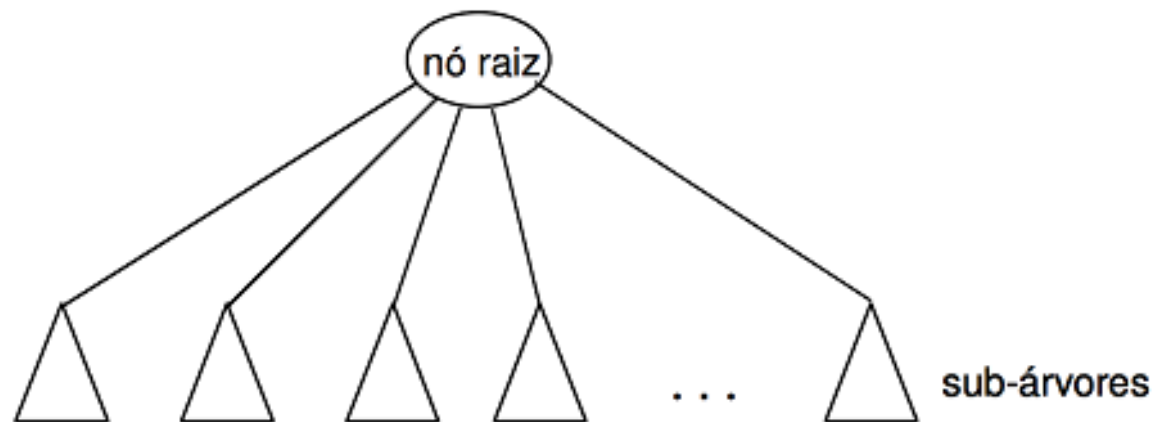
- Árvores
 - São definidas como um conjunto de vértices (ou “nós”) e “arestas” que satisfazem certos requisitos
- Vértices
 - É cada uma das entidades representadas na árvore (depende da natureza do problema)
- Arestas
 - É uma conexão entre dois vértices



Árvores

Um conjunto de nós tal que:

- existe um nó r , denominado **raiz**, com zero ou mais sub-árvores, cujas raízes estão ligadas a r .



Árvores

- os nós raízes destas sub-árvores são os **filhos** de r .
- os **nós internos** da árvore são os nós com filhos.
- as **folhas** ou **nós externos** da árvore são os nós sem filhos.

Árvore

- **Pai**

- É o antecessor imediato de um vértice

- **Filhos**

- É o sucessor imediato de um vértice

- **Raiz**

- É o vértice que não possui pai

- Nós terminais ou **folhas**

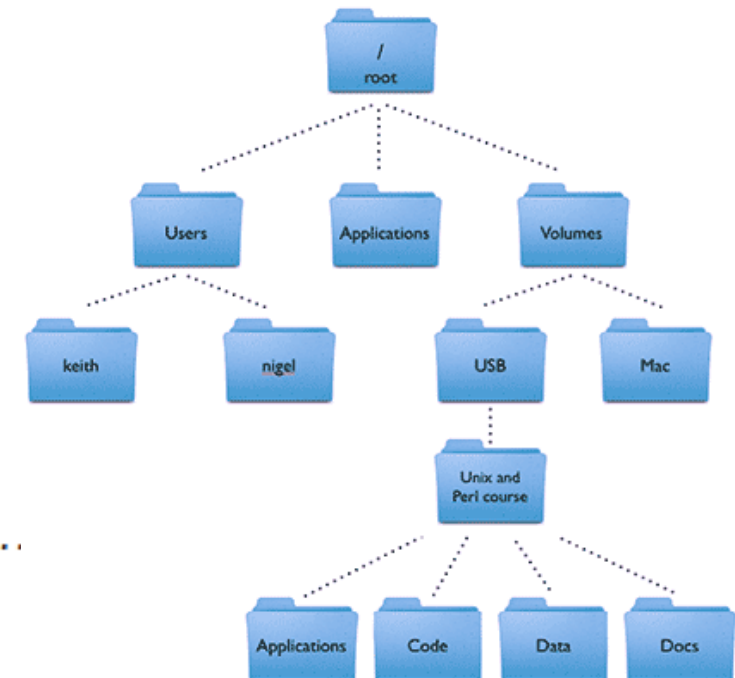
- É o vértice que não possui filhos

- Nós Não-Terminais ou **Internos**

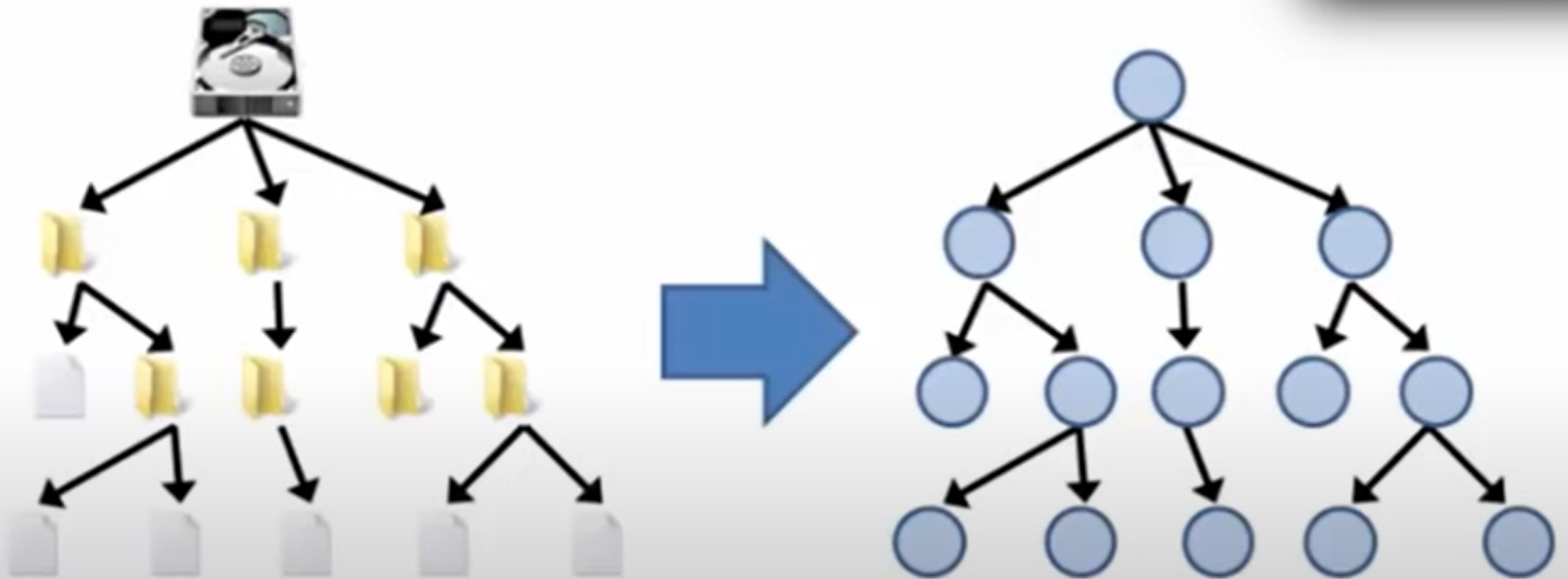
- Qualquer vértice que possui pelo menos 1 filho

Árvores

- Estrutura de diretórios e arquivos de um sistema operacional
- Análise semântica de equações matemáticas
- *É uma estrutura usada por vários outros algoritmos...*

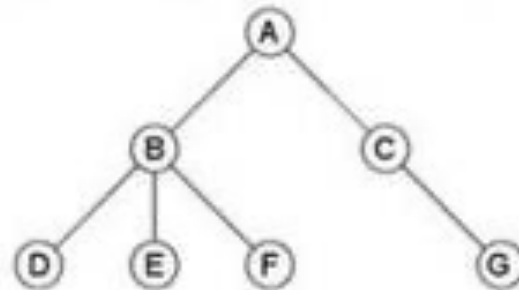


Árvores

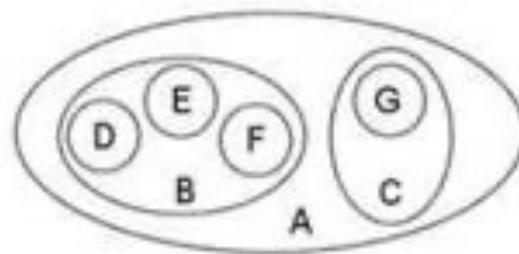


Árvores

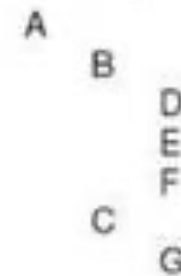
- Grafo (representação mais utilizada)



- Diagrama de Venn (ou digrama de inclusão)



- Identação



- Parênteses Aninhados

(A (B(D, E, F), C(G)))

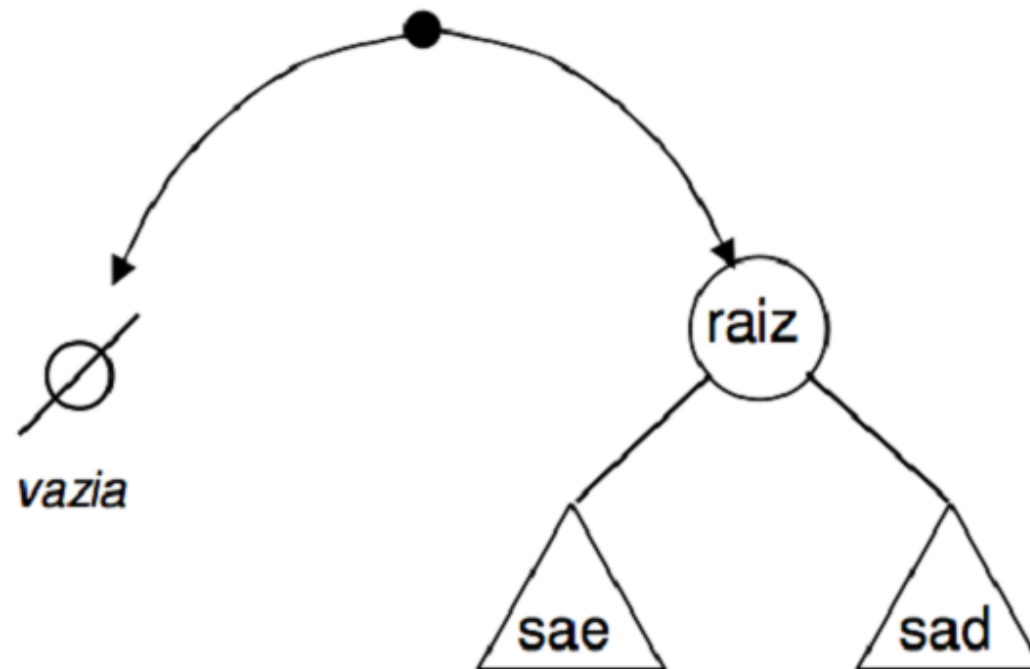
Árvores

- Existem diferentes tipos de árvores em computação, desenvolvidas para diferentes tipos de aplicação:
 - Árvores binária de busca
 - Árvore AVL
 - Árvore Rubro-Negra
 - Árvore B+
 - Árvore 2-3
 - Árvore 2-3-4
 - Quadtree
 - Octree

Árvore Binária

- uma árvore em que cada nodo tem zero, um ou dois filhos.
- uma árvore binária é:
 - ★ uma árvore vazia, ou;
 - ★ um nodo raiz com duas sub-árvores: a sub-árvore da direita (sad); a sub-árvore da esquerda (sae).

Árvore Binária

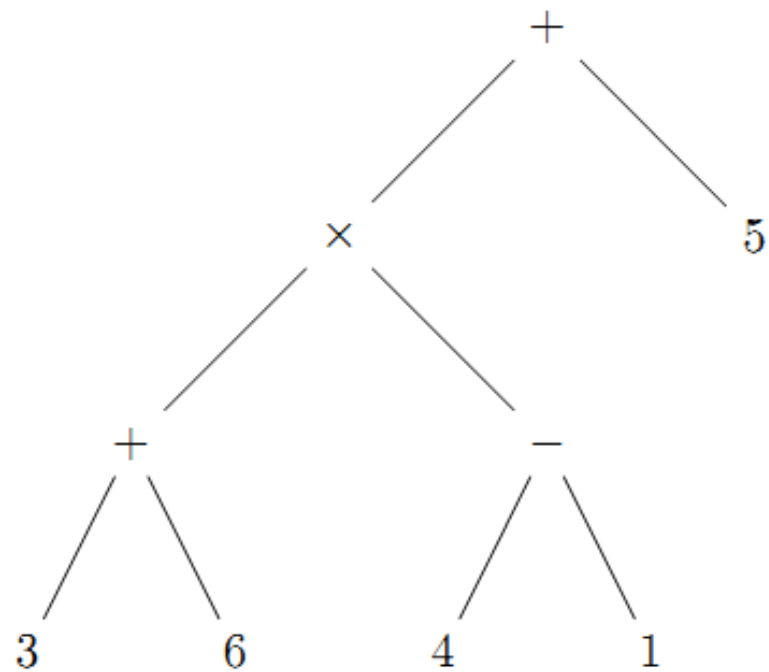


Árvore Binária

Árvore binária representando expressões aritméticas:

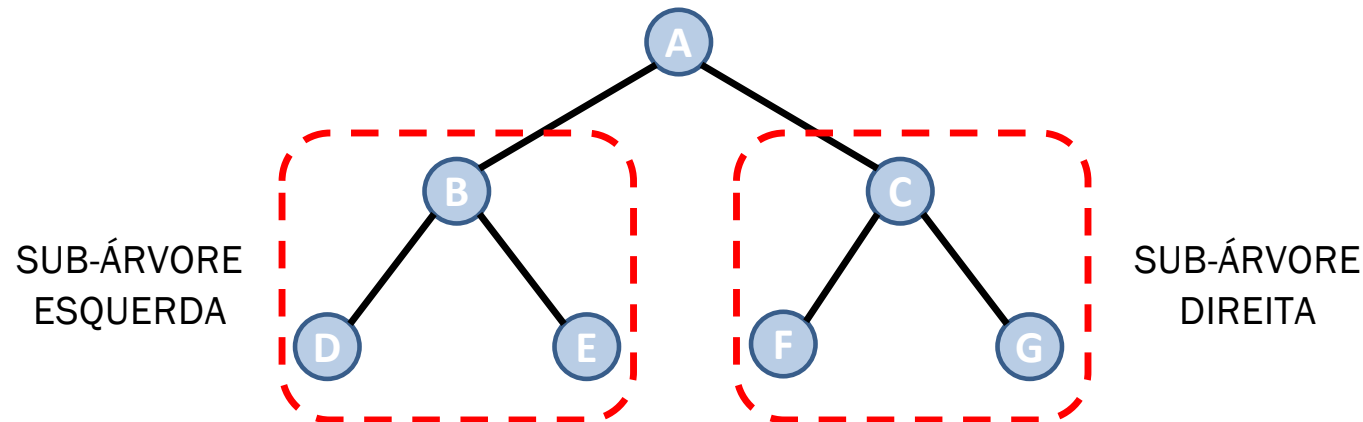
- nós folhas representam operandos
- nós internos operadores
- exemplo: $(3 + 6) \times (4 - 1) + 5$

Árvore Binária



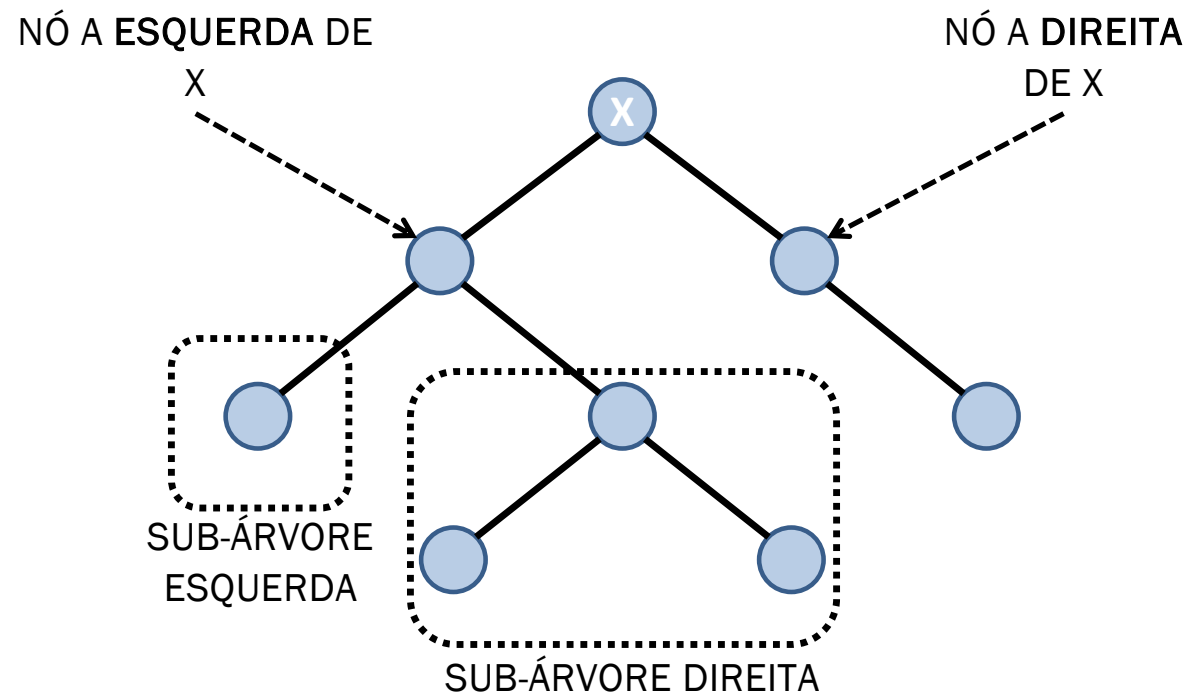
Árvore Binária

- É um tipo especial de árvore
 - Cada nó pode possuir nenhuma, uma ou no máximo duas **sub-árvores**
 - Sub-árvore da **esquerda** e a da **direita**
 - Usadas em situações onde, a cada passo, é preciso tomar uma decisão entre duas direções



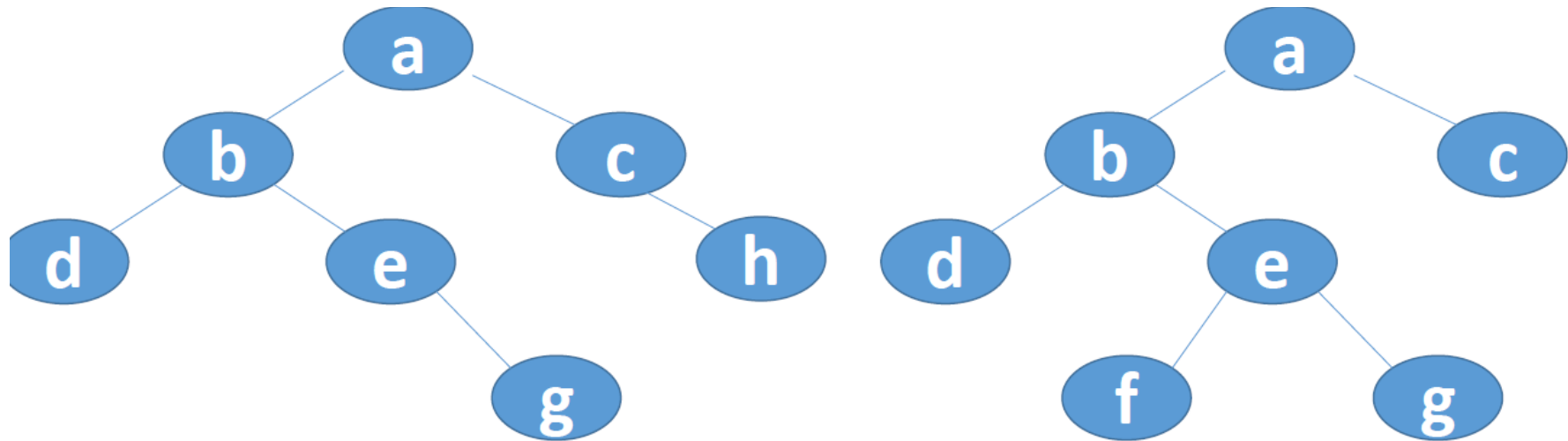
Árvore Binária

- Exemplo de árvore binária



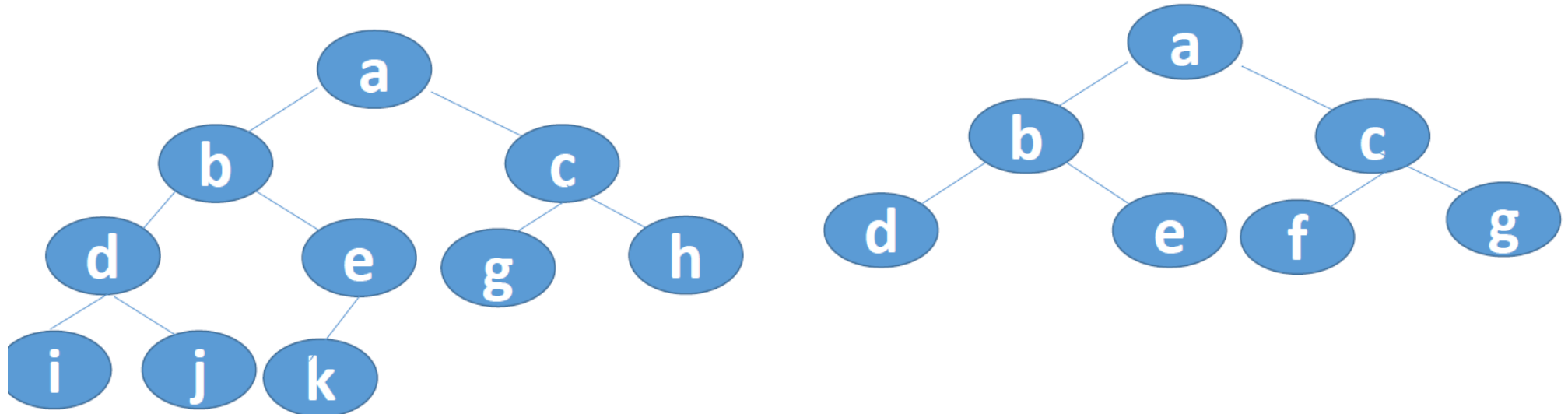
Árvore Binária

Uma árvore binária é **completa**, se todo elemento não terminal tem associado exatamente duas subárvores não vazias. Isto é, ou este elemento tem as duas subárvores ou não tem nenhuma.



Árvore Binária

Uma árvore binária é **cheia**, se é **completa** e todas as folhas estão no mesmo nível. Ou seja, todos os nós internos tem duas subárvores associadas. E uma árvore é dita **quase cheia** se está cheia até o penúltimo nível e todas as folhas do seguinte nível, estão o mais a esquerda quanto possível.



Árvore Binária

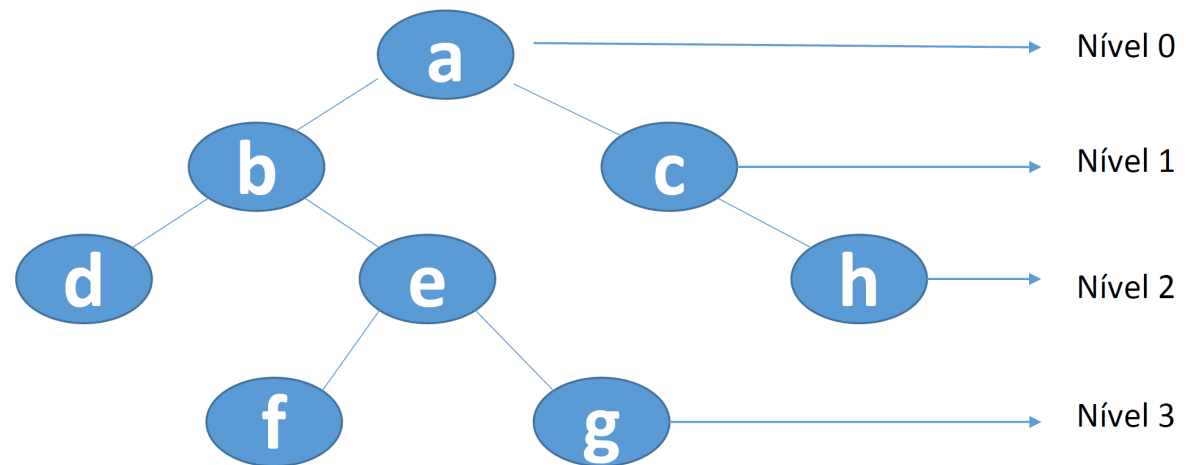
- **Propriedade fundamental de árvores:** Nem sempre existe um caminho entre dois elementos de uma árvore, mas se existe, este caminho é único.
- Um caminho entre dois elementos e_1 e e_2 é uma sequência $\langle x_1, x_2, \dots, x_n \rangle$, onde $e_1 = x_1$, é o primeiro elemento, $e_2 = x_n$ é o último elemento, e cada elemento é pai de seu sucessor.
- A **longitude** de um caminho $\langle x_1, x_2, \dots, x_n \rangle$ é $N-1$, ou seja, o número de vezes que se aplica a relação pai/filho.
- Sempre existe uma caminho de longitude 0 que vai de um elemento r para si mesmo e corresponde a sequência $\langle r \rangle$.
- O caminho que parte da raiz e termina numa folha é chamado de **galho** ou ramo da árvore.

Árvore Binária

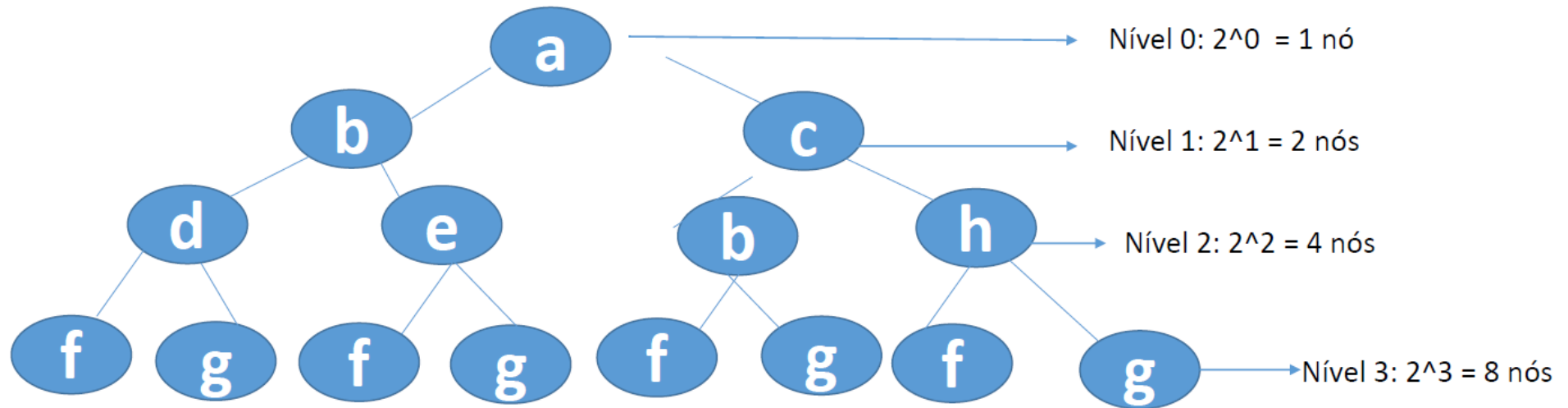
- A **altura (h)** de uma árvore binária é a longitude do maior caminho. Ou o número de vezes que se aplica a relação pai/filho no maior caminho (igual ao número de ramos/galhos).
- A altura de uma folha é 0. Pode-se considerar que a altura de uma árvore vazia é -1, para facilitar na implementação.
- O **peso** de uma árvore é o número de elementos desta árvore. Recursivamente se pode definir como a soma dos pesos das subárvores mais 1.
- De acordo com a definição, o peso de uma árvore vazia é zero.
- O nível de um elemento dentro da árvore binária se define como a longitude do caminho que parte da raiz e chega até esse elemento. Desta forma, o nível da raiz é 0, e o nível de qualquer elemento é o nível de seu pai mais 1.

Árvore Binária

- A altura da árvore é 3, que coincide com a altura do nó a.
- O peso da árvore é 8.
- O elemento “a” é a raiz da árvore.
- A altura de (d,f,g,h) é 0, de (e,c) é 1, de (b é 2) e de a é 3.



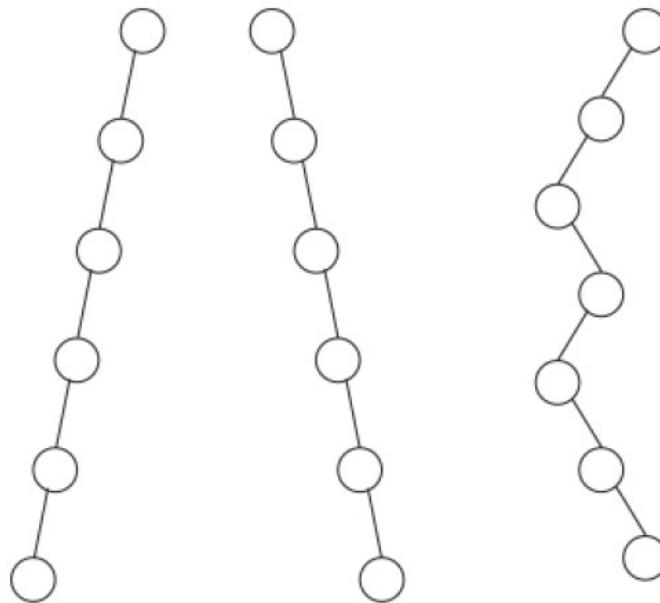
Árvore Binária - Altura



O número de N de nós de uma árvore cheia de altura h é: $N = 2^{(h+1)} - 1$

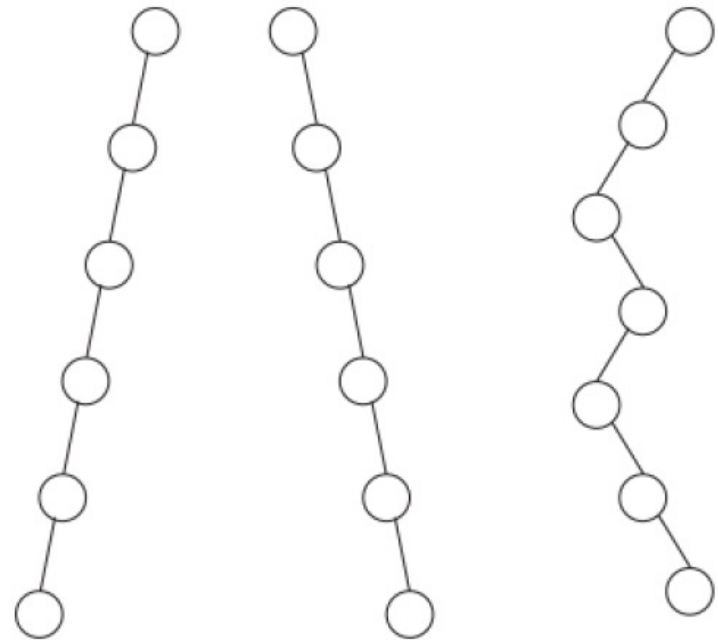
Árvore Binária - Altura

Se a árvore está degenerada (os nós internos tem somente uma subárvore) o número de N de nós de uma árvore degenerada de altura h é: $N - 1 = h$



Árvore Binária - Altura

- A relação entre a altura de uma árvore e seu número de nós é um dado importante para várias aplicações.
- A árvore binária que possui altura máxima é aquela cujos nós interiores possuem exatamente uma subárvore vazia, denominada zigue-zague (árvore mais a direita da imagem).
- Naturalmente a altura de uma árvore zigue-zague é igual a N .



Árvore Binária - Altura

- Uma árvore completa sempre apresenta uma altura mínima.
- Seja T uma árvore binária completa com $N > 0$ nós. Então T possui altura mínima.
- Além disso, $h = \text{piso de } \log N$

Árvore Binária - Altura

- Esforço computacional necessário para alcançar qualquer nó da árvore
 - proporcional à altura da árvore
 - altura de uma árvore binária com n nós
 - mínima: proporcional a $\log n$ (caso da árvore cheia)
 - máxima: proporcional a n (caso da árvore degenerada)

Árvore Binária

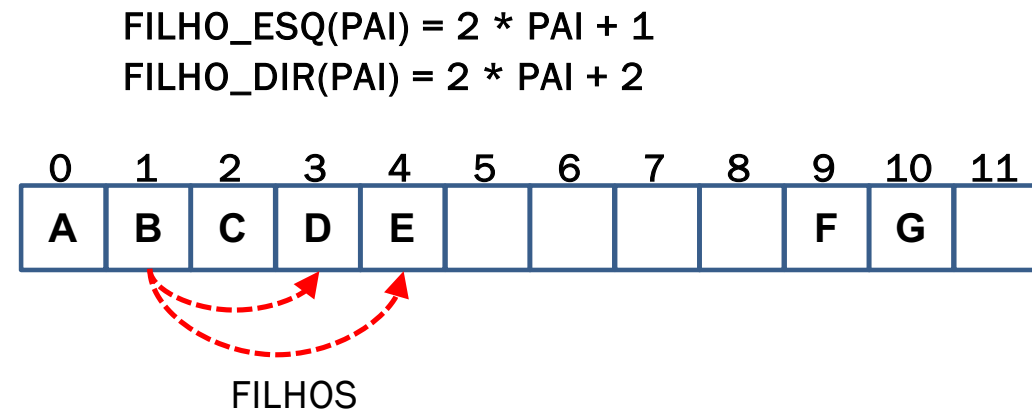
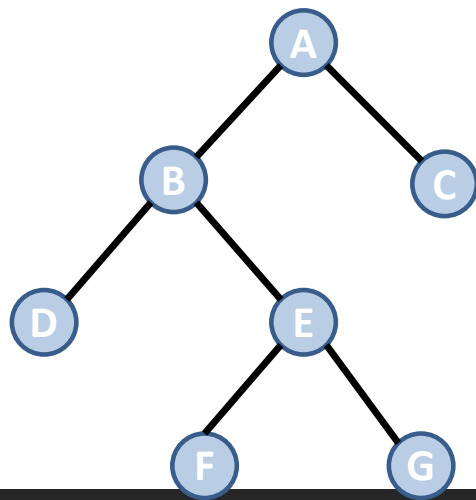
- Duas árvores binárias são iguais se ambas são vazias, ou se suas raízes são iguais, e o mesmo acontece para suas respectivas subárvores esquerda e direita.
- Duas árvores são isomorfas se têm a mesma forma(estrutura), mas não necessariamente os mesmos elementos.
- Duas árvores são semelhantes se têm os mesmos elementos, ainda que não sejam isomorfas.

Tipo de representação

- Como implementar uma árvore no computador?
- Existem duas abordagens muito utilizadas
 - Usando um array (alocação estática)
 - Usando uma lista encadeada (alocação dinâmica)

Tipo de representação

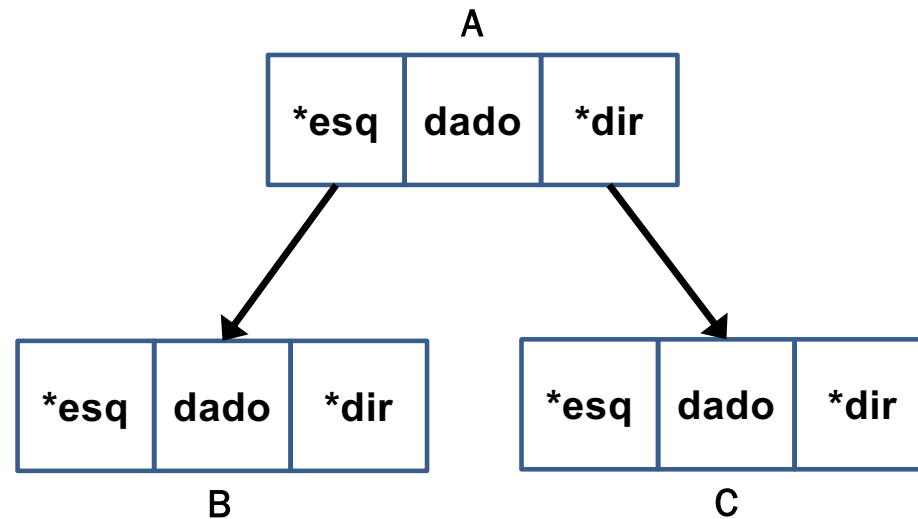
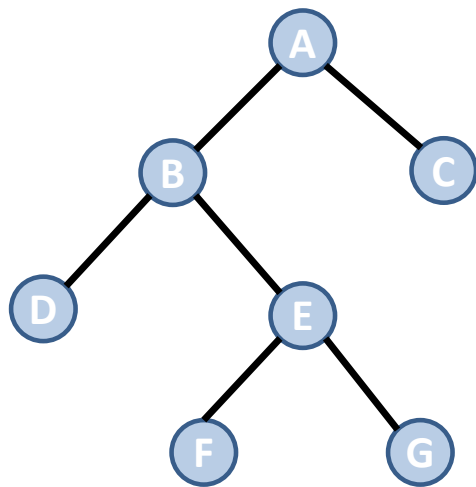
- Usando um array (alocação estática)
 - Necessário definir o número máximo de nós
 - Tamanho do array
 - Usa 2 funções para retornar a posição dos filhos à esquerda e à direita de um pai



Tipo de representação

Lista encadeada (alocação dinâmica)

- Espaço de memória alocado em tempo de execução
 - A árvore cresce à medida que novos elementos são armazenados, e diminui à medida que elementos são removidos



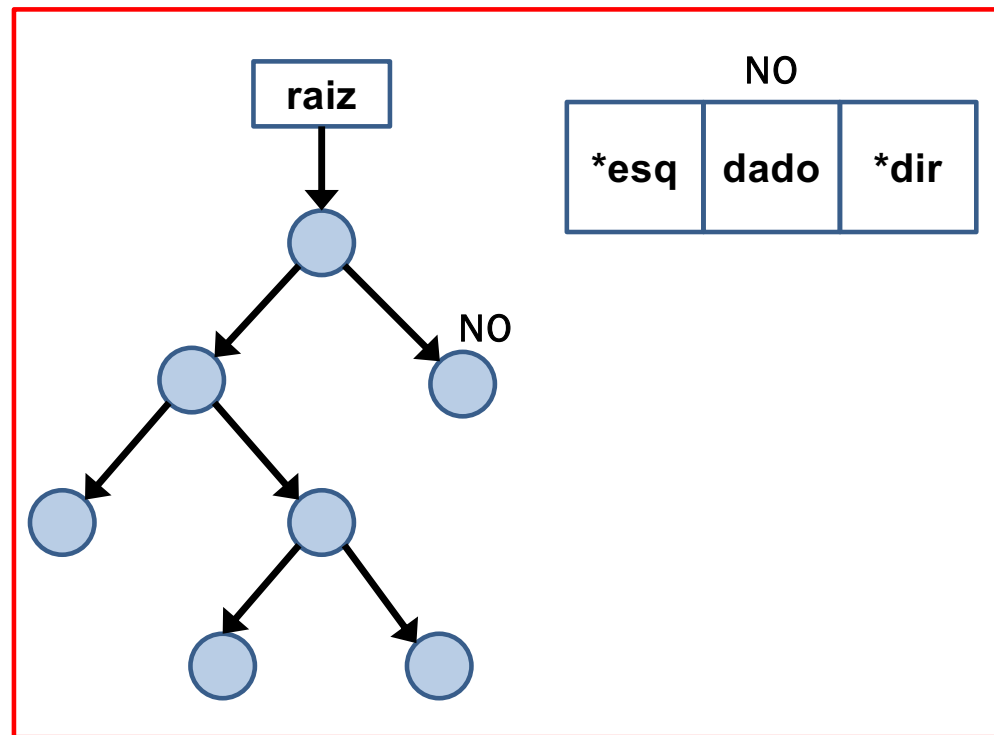
Árvore Binária

Definição

- Uso de alocação dinâmica
 - Para guardar o primeiro nó da árvore utilizamos um **ponteiro para ponteiro**
 - Um **ponteiro para ponteiro** pode guardar o endereço de um **ponteiro**
 - Assim, fica fácil mudar quem é a **raiz** da árvore (se necessário)

Árvore Binária - Definição

ArvBin* raiz



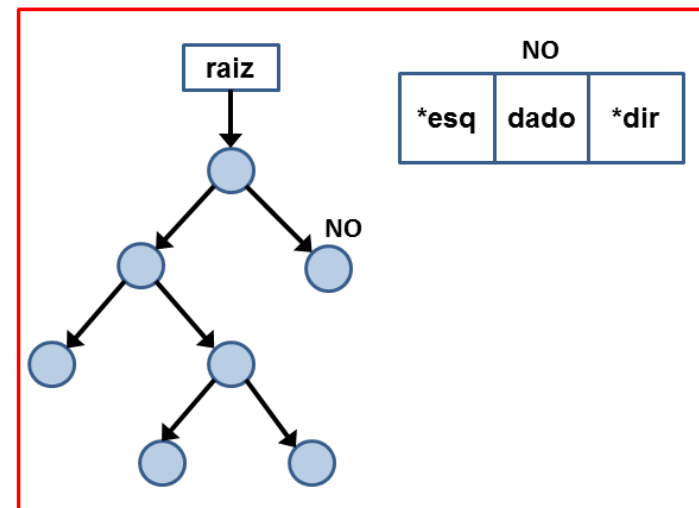
Árvore Binária - Definição

```
//Arquivo ArvoreBinaria.h
typedef struct NO* ArvBin;

//Arquivo ArvoreBinaria.c
#include <stdio.h>
#include <stdlib.h>
#include "ArvoreBinaria.h" //inclui os Protótipos
struct NO{
    int info;
    struct NO *esq;
    struct NO *dir;
};

//programa principal
ArvBin* raiz; //ponteiro para ponteiro
```

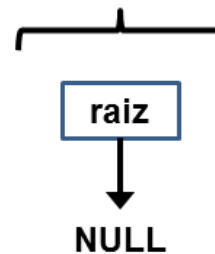
ArvBin* raiz



Árvore Binária - Criando a árvore

```
1 //arquivo ArvoreBinaria.h
2 ArvBin* cria_ArvBin();
3
4 //arquivo ArvoreBinaria.c
5 ArvBin* cria_ArvBin(){
6     ArvBin* raiz = (ArvBin*) malloc(sizeof(ArvBin));
7     if(raiz != NULL)
8         *raiz = NULL;
9     return raiz;
10 }
11
12 //programa principal
13 ArvBin* raiz = cria_ArvBin();
```

ArvBin* raiz

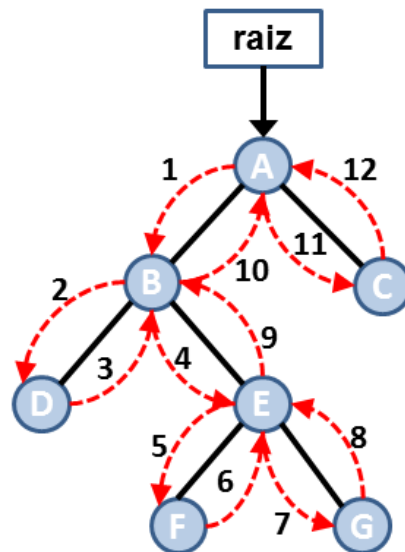


Árvore Binária - Liberando

- Uso de 2 funções: uma percorre e libera os nós, outra trata a raiz

```
5 void libera_NO(struct NO* no) {  
6     if(no == NULL)  
7         return;  
8     libera_NO(no->esq);  
9     libera_NO(no->dir);  
10    free(no);  
11    no = NULL;  
12 }  
13 void libera_ArvBin(ArvBin* raiz) {  
14     if(raiz == NULL)  
15         return;  
16     libera_NO(*raiz); //libera cada nó  
17     free(raiz); //libera a raiz  
18 }
```

Árvore Binária -Remoção: passo a passo



	1	visita B
	2	visita D
✗	3	libera D, volta para B
	4	visita E
	5	visita F
✗	6	libera F, volta para E
	7	visita G
✗	8	libera G, volta para E
✗	9	libera E, volta para B
✗	10	libera B, volta para A
	11	visita C
✗	12	libera C, volta para A e
✗		libera A
✗		libera raiz

Árvore Binária

Informações básicas sobre a árvore

■ Altura

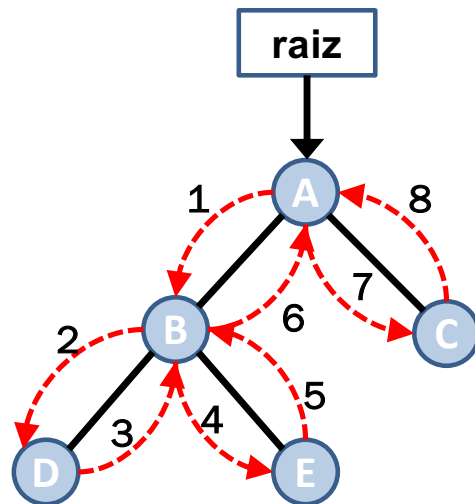
- Número total de níveis de uma árvore

```
5 int altura_ArvBin(ArvBin *raiz){
6     if (raiz == NULL)
7         return 0;
8     if (*raiz == NULL)
9         return 0;
10    int alt_esq = altura_ArvBin(&((*raiz)->esq));
11    int alt_dir = altura_ArvBin(&((*raiz)->dir));
12    if (alt_esq > alt_dir)
13        return (alt_esq + 1);
14    else
15        return (alt_dir + 1);
16 }
```

Árvore Binária

Informações básicas sobre a árvore

■ Altura



	inicia no nó A
1	visita B
2	visita D
3	D é nó folha: altura é 1. Volta para B
4	visita E
5	E é nó folha: altura é 1. Volta para B
6	altura de B é 2: maior altura dos filhos + 1. Volta para A
7	visita C
8	C é nó folha: altura é 1. Volta para A
	altura de A é 3: maior altura dos filhos + 1.

TAD Árvore Binária

Informações básicas sobre a árvore

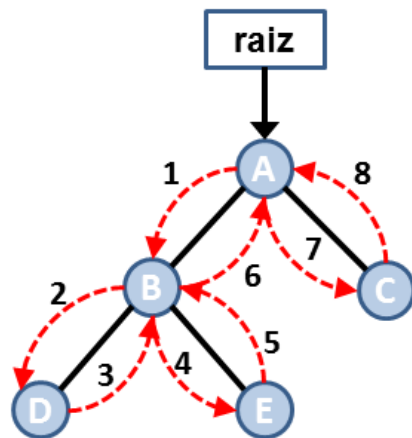
- Número de nós
 - Quantidade de elementos na árvore

```
5 int totalNO_ArvBin(ArvBin *raiz) {  
6     if (raiz == NULL)  
7         return 0;  
8     if (*raiz == NULL)  
9         return 0;  
10    int alt_esq = totalNO_ArvBin(&((*raiz)->esq));  
11    int alt_dir = totalNO_ArvBin(&((*raiz)->dir));  
12    return(alt_esq + alt_dir + 1);  
13 }
```

TAD Árvore Binária

Informações básicas sobre a árvore

- Número de nós
 - Quantidade de elementos na árvore



1	visita B
2	visita D
3	D é nó folha: conta como 1 nó. Volta para B
4	visita E
5	E é nó folha: conta como 1 nó. Volta para B
6	Número de nós em B é 3: total de nós a direita (1) + total de nós a esquerda (1) + 1. Volta para A
7	visita C
8	C é nó folha: conta como 1 nó. Volta para A
Número de nós em A é 5: total de nós a direita (1) + total de nós a esquerda (3) + 1.	

Percurso na árvore

Percorrer todos os nós é uma operação muito comum em árvores binárias

- Cada nó é visitado uma única vez
 - Isso gera uma sequência linear de nós, cuja ordem depende de como a árvore foi percorrida
- Não existe uma **ordem natural** para se percorrer todos os nós de uma árvore binária
- Isso pode ser feito para executar alguma ação em cada nó
 - Essa ação pode ser mostrar (imprimir) o valor do nó, modificar esse valor, etc.

Percurso na árvore

Podemos percorrer a árvore de 3 formas

- Percurso pré-ordem
 - visita a **raiz**, o filho da **esquerda** e o filho da **direita**
- Percurso um-ordem
 - visita o filho da **esquerda**, a **raiz** e o filho da **direita**
- Percurso pós-ordem
 - visita o filho da **esquerda**, o filho da **direita** e a **raiz**

Esses são os percursos mais importantes

- Existem outras formas de percurso

Percurso pré-ordem

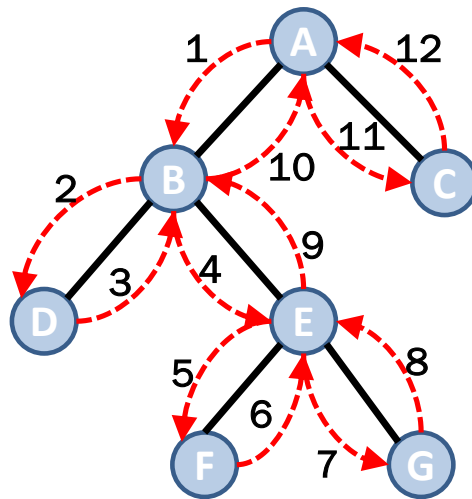
- Ordem de visitação
 - Raiz
 - Filho esquerdo
 - Filho direito

```
5 void preOrdem_ArvBin(ArvBin *raiz) {  
6     if (raiz == NULL)  
7         return;  
8     if (*raiz != NULL) {  
9         printf("%d\n", (*raiz)->info);  
10        preOrdem_ArvBin(&((*raiz)->esq));  
11        preOrdem_ArvBin(&((*raiz)->dir));  
12    }  
13 }
```

Percurso pré-ordem

■ Ordem de visitação

- Raiz
- Filho esquerdo
- Filho direito



RESULTADO: ABDEFGC

	inicia no nó A
1	imprime A, visita B
2	imprime B, visita D
3	imprime D, volta para B
4	visita E
5	imprime E, visita F
6	imprime F, volta para E
7	visita G
8	imprime G, volta para E
9	volta para B
10	volta para A
11	visita C
12	imprime C, volta para A

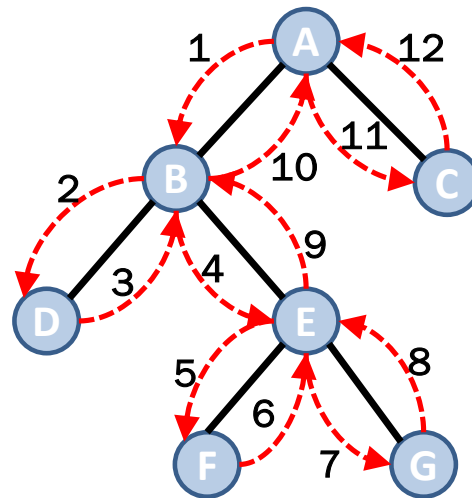
Percurso em-ordem

- Ordem de visitação
 - Filho esquerdo
 - Raiz
 - Filho direito

```
5 void emOrdem_ArvBin(ArvBin *raiz) {  
6     if(raiz == NULL)  
7         return;  
8     if(*raiz != NULL) {  
9         emOrdem_ArvBin(&((*raiz)->esq));  
10        printf("%d\n", (*raiz)->info);  
11        emOrdem_ArvBin(&((*raiz)->dir));  
12    }  
13 }
```

Percurso em-ordem

- Ordem de visitação
 - Filho esquerdo
 - Raiz
 - Filho direito



RESULTADO: DBFEGAC

	inicia no nó A
1	visita B
2	visita D
3	imprime D, volta para B
4	imprime B, visita E
5	visita F
6	imprime F, volta para E
7	imprime E, visita G
8	imprime G, volta para E
9	volta para B
10	volta para A
11	imprime A, visita C
12	imprime C, volta para A

Percurso pós-ordem

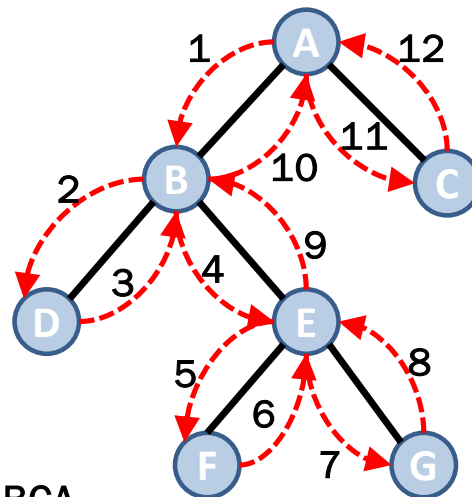
- Ordem de visitação
 - Filho esquerdo
 - Filho direito
 - Raiz

```
5 void posOrdem_ArvBin(ArvBin *raiz) {  
6     if(raiz == NULL)  
7         return;  
8     if(*raiz != NULL) {  
9         posOrdem_ArvBin(&((*raiz)->esq));  
10        posOrdem_ArvBin(&((*raiz)->dir));  
11        printf("%d\n", (*raiz)->info);  
12    }  
13 }
```

Percurso pós-ordem

■ Ordem de visitação

- Filho esquerdo
- Filho direito
- Raiz



RESULTADO: DFGEBCA

	inicia no nó A
1	visita B
2	visita D
3	imprime D, volta para B
4	visita E
5	visita F
6	imprime F, volta para E
7	visita G
8	imprime G, volta para E
9	imprime E, volta para B
10	imprime B, volta para A
11	visita C
12	imprime C, volta para A e imprime A

Árvore Binária de Busca - ABB

Árvore Binária de Busca

Definição

- É uma árvore binária
 - Cada nó pode ter 0, 1 ou 2 filhos
- Cada nó possui da árvore possui um valor (chave) associado a ele
 - Não existem valores repetidos
- Esse valor determina a posição do nó na árvore

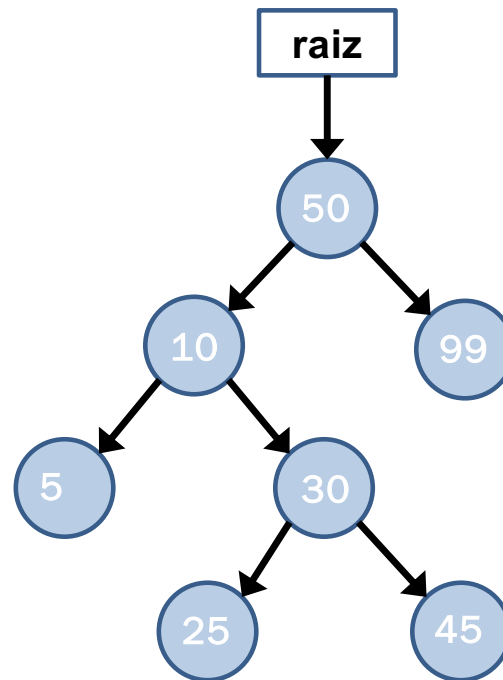
Árvore Binária de Busca

Regra para posicionamento dos valores na árvore

- Para cada nó pai
 - todos os valores da sub-árvore **esquerda** são menores do que o nó pai
 - todos os valores da sub-árvore **direita** são maiores do que o nó pai;
- Inserção e remoção devem ser realizadas respeitando essa regra de posicionamento dos nós.

Árvore Binária de Busca

Exemplo



Árvore Binária de Busca

Ótima alternativa para operações de busca binária

- Possui a vantagem de ser uma estrutura dinâmica em comparação ao array
- É mais fácil inserir valores na árvore do que em um array ordenado
 - Array: envolve deslocamento de elementos

Árvore Binária de Busca - Inserção

Para inserir um valor **V** na árvore

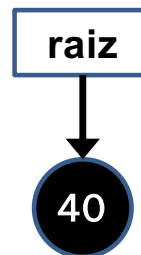
- Se a raiz é igual a **NULL**, insira o nó
- Se **V** é menor do que a raiz: vá para a **sub-árvore esquerda**
- Se **V** é maior do que a raiz: vá para a **sub-árvore direita**
- Aplique o método **recursivamente**
 - pode ser feito sem recursão

Dessa forma, percorremos um conjunto de nós da árvore até chegar ao nó folha que irá se tornar o pai do novo nó

Árvore Binária de Busca: Inserção

Devemos também considerar a inserção em uma árvore que está vazia

**Insere o valor '40'
em uma árvore
vazia**



`*raiz = novo;`

TAD Árvore Binária

Inserção em uma Árvore Binária de Busca

```
int insere_ArvBin(ArvBin* raiz, int valor) {
    if(raiz == NULL)
        return 0;
    struct NO* novo;
    novo = (struct NO*) malloc(sizeof(struct NO));
    if(novo == NULL)
        return 0;
    novo->infor = valor;
    novo->dir = NULL;
    novo->esq = NULL;
    //procurar onde inserir!
```

TAD Árvore Binária

Inserção em uma Árvore Binária de Busca

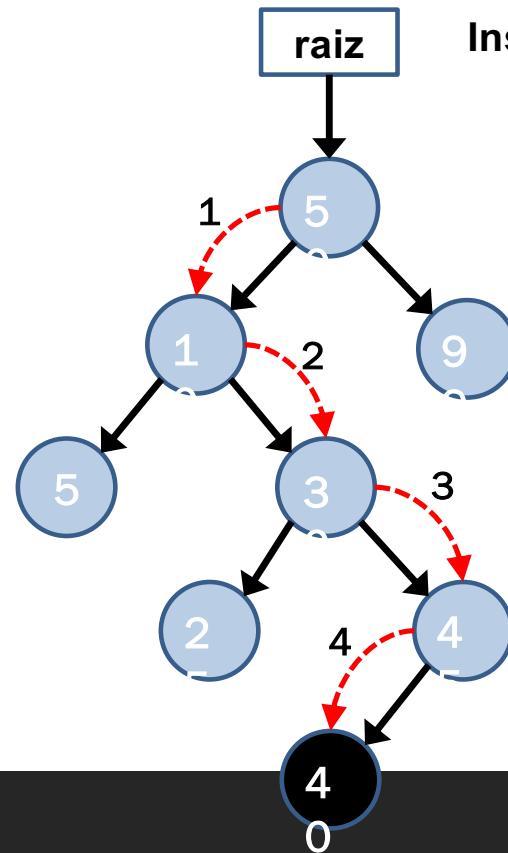
Navega nos nós
da árvore até
chegar em um
nó folha

Insere como
filho desse nó
folha

```
if(*raiz == NULL)
    *raiz = novo;
else{
    struct NO* atual = *raiz;
    struct NO* ant = NULL;
    while(atual != NULL){
        ant = atual;
        if(valor == atual->info){
            free(novo);
            return 0; //elemento já existe
        }
        if(valor > atual->info)
            atual = atual->dir;
        else
            atual = atual->esq;
    }
    if(valor > ant->info)
        ant->dir = novo;
    else
        ant->esq = novo;
}
return 1;
```

Árvore Binária de Busca: Inserção

Exemplo



Inserir o valor '40' como um nó folha

1	valor é menor do que 50: visita filho da esquerda
2	valor é maior do que 10: visita filho da direita
3	valor é maior do que 30: visita filho da direita
4	valor é menor do que 45: visita filho da esquerda
Fim	Não existe filho da esquerda. Valor passa a ser o filho da esquerda de 45

Árvore Binária de Busca: Busca

Consultar se um determinado nó **V** existe em uma árvore é similar a operação de inserção

- primeiro compare o valor buscado com a **raiz**;
- se **V** é menor do que a raiz: vá para a **sub-árvore da esquerda**;
- se **V** é maior do que a raiz: vá para a **sub-árvore da direita**;
- aplique o método recursivamente até que a raiz seja igual ao valor buscado
 - pode ser feito sem recursão

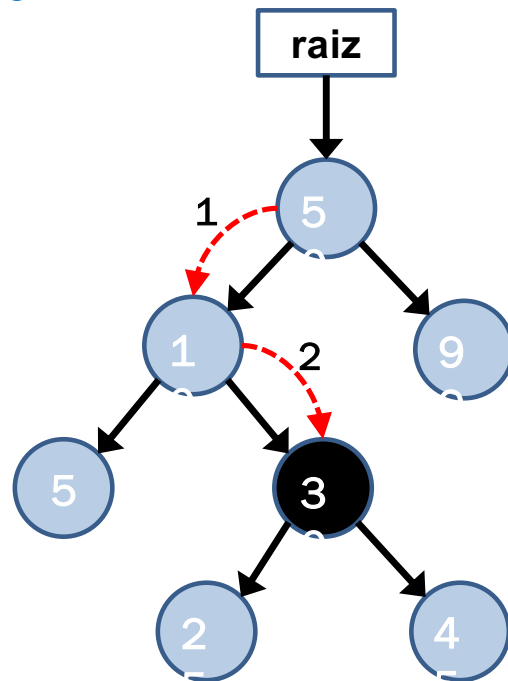
TAD Árvore Binária

Busca em uma Árvore Binária de Busca

```
6 int consulta_ArvBin(ArvBin *raiz, int valor){
7     if(raiz == NULL)
8         return 0;
9     struct NO* atual = *raiz;
10    while(atual != NULL){
11        if(valor == atual->info){
12            return 1;
13        }
14        if(valor > atual->info)
15            atual = atual->dir;
16        else
17            atual = atual->esq;
18    }
19    return 0;
20 }
```

Árvore Binária de Busca: Busca

Exemplo

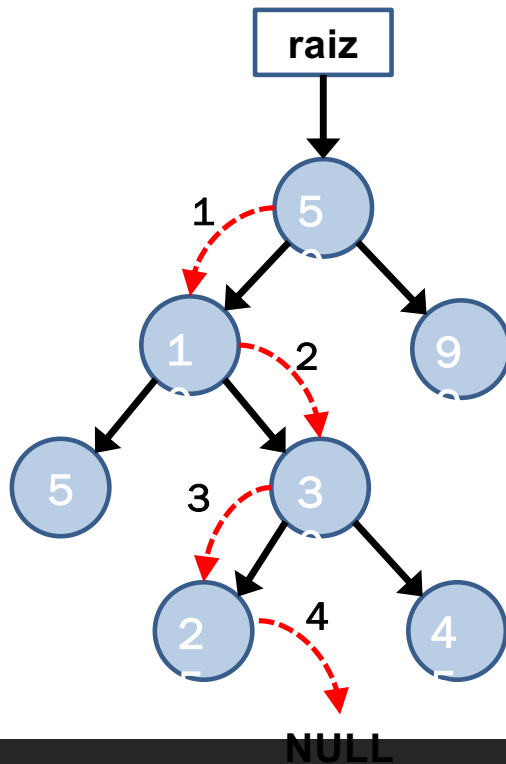


Valor procurado: 30

1	valor procurado é menor do que 50: visita filho da esquerda
2	valor procurado é maior do que 10: visita filho da direita
Fim	valor procurado é igual ao do nó: retornar dados do nó

Árvore Binária de Busca: Busca

Exemplo: valor buscado não existe!



Valor procurado: 28

1	valor procurado é menor do que 50: visita filho da esquerda
2	valor procurado é maior do que 10: visita filho da direita
3	valor procurado é menor do que 30: visita filho da esquerda
4	valor procurado é maior do que 25: visita filho da direita
Fim	Filho da direita de 25 não existe: a busca falhou

Árvore Binária de Busca: Remoção

Remover um nó de uma árvore binária de busca não é uma tarefa tão simples quanto a inserção.

- Isso ocorre porque precisamos procurar o nó a ser removido da árvore o qual pode ser um
 - nó folha
 - nó interno (que pode ser a raiz), com um ou dois filhos.
- Se for um nó interno
 - Reorganizar a árvore para que ela continue sendo uma árvore binária de busca

TAD Árvore Binária

Remoção em uma Árvore Binária de Busca

- Trabalha com 2 funções
 - Busca pelo nó
 - Tratar os 3 tipos de remoção: com 0, 1 ou 2 filhos

```
8  int remove_ArvBin(ArvBin *raiz, int valor){
9      /*
10     FUNÇÃO RESPONSÁVEL PELA BUSCA
11     DO NÓ A SER REMOVIDO
12     */
13 }
14 struct NO* remove_atual(struct NO* atual){
15     /*
16     FUNÇÃO RESPONSÁVEL POR TRATAR OS 3
17     TIPOS DE REMOÇÃO
18     */
19 }
```

TAD Árvore Binária

Remoção em uma Árvore Binária de Busca

Achou o nó a ser
removido. Tratar o
tipo de remoção

Continua andando
na árvore a procura
do nó a ser
removido

```
int remove_ArvBin(ArvBin *raiz, int valor) {
    if(raiz == NULL) return 0;
    struct NO* ant = NULL;
    struct NO* atual = *raiz;
    while(atual != NULL) {
        if(valor == atual->info) {
            if(atual == *raiz)
                *raiz = remove_atual(atual);
            else {
                if(ant->dir == atual)
                    ant->dir = remove_atual(atual);
                else
                    ant->esq = remove_atual(atual);
            }
            return 1;
        }
        ant = atual;
        if(valor > atual->info)
            atual = atual->dir;
        else
            atual = atual->esq;
    }
    return 0;
}
```

TAD Árvore Binária

Remoção em uma Árvore Binária de Busca

```
1 struct NO* remove_atual(struct NO* atual) {
2     struct NO *no1, *no2;
3     if(atual->esq == NULL) {
4         no2 = atual->dir;
5         free(atual);
6         return no2;
7     }
8     no1 = atual;
9     no2 = atual->esq;
10    while(no2->dir != NULL) {
11        no1 = no2;
12        no2 = no2->dir;
13    }
14
15    if(no1 != atual) {
16        no1->dir = no2->esq;
17        no2->esq = atual->esq;
18    }
19    no2->dir = atual->dir;
20    free(atual);
21    return no2;
22 }
```

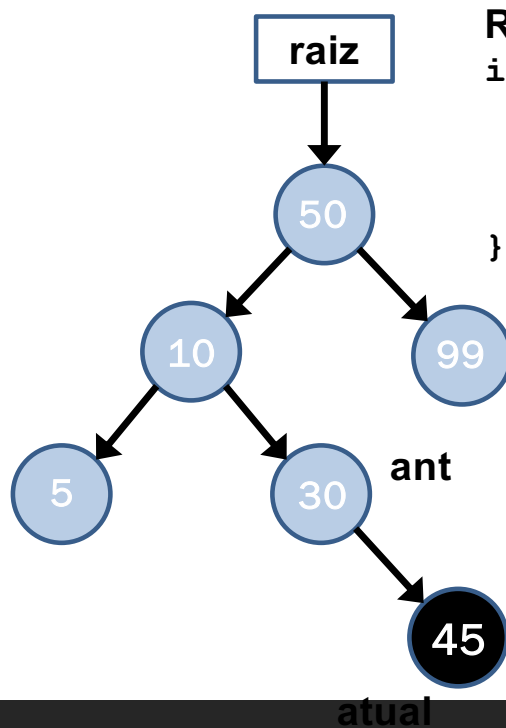
Sem filho da esquerda.
Apontar para o filho da
direita (trata nó folha e nó
com 1 filho)

Procura filho mais a
direita na sub-árvore
da esquerda.

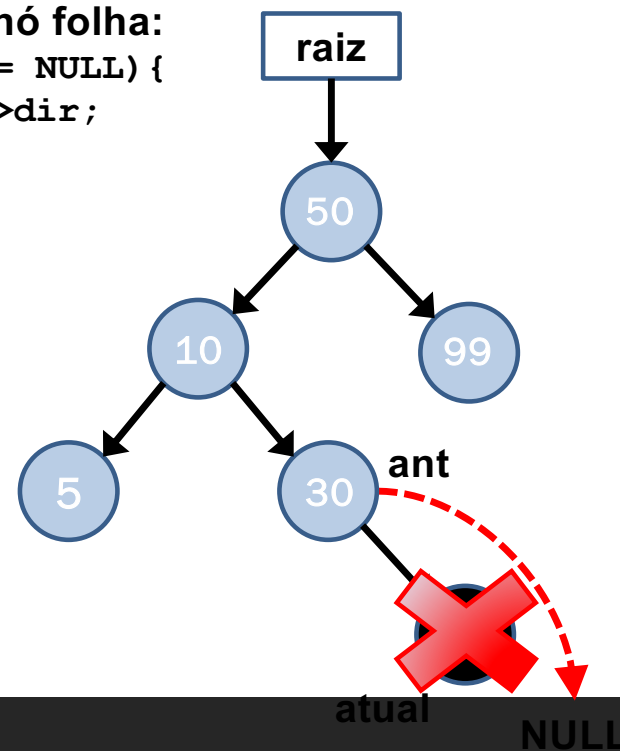
Copia o filho mais a
direita na sub-árvore
da esquerda para o
lugar do nó removido.

Árvore Binária de Busca: Remoção

Exemplo: remoção de um nó folha

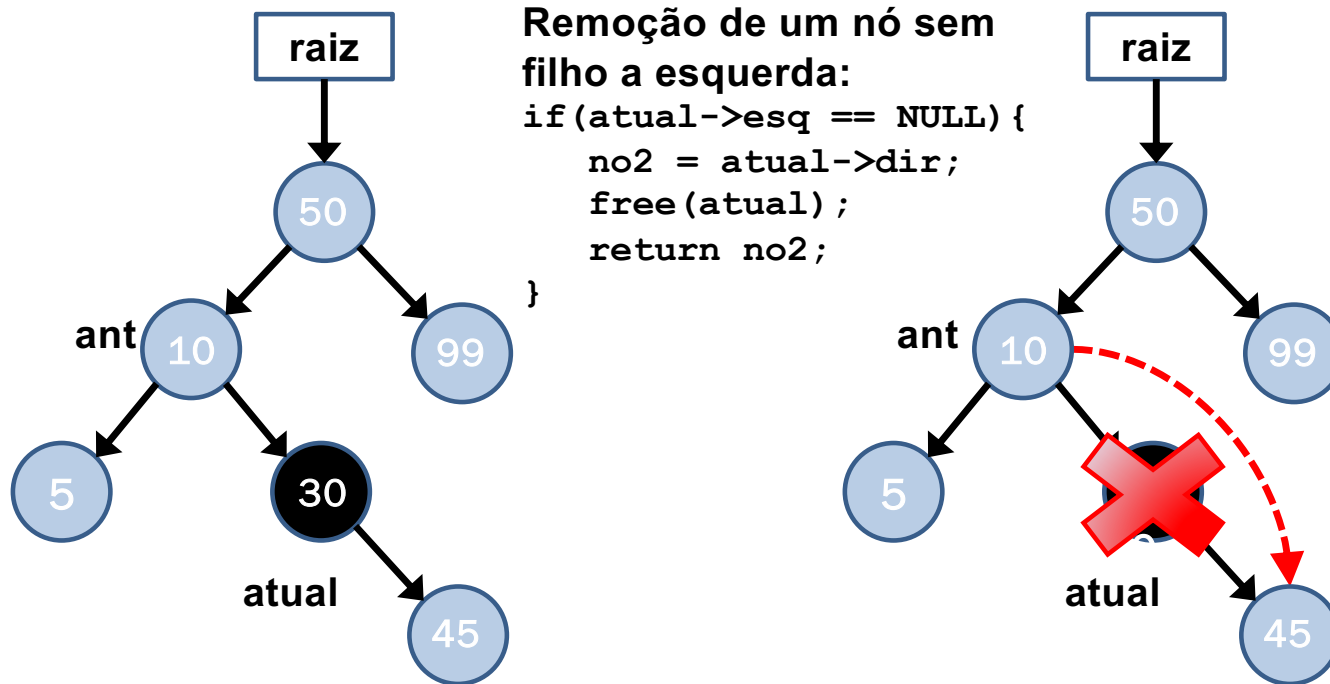


Remoção de um nó folha:
`if(atual->esq == NULL){
 no2 = atual->dir;
 free(atual);
 return no2;
}`



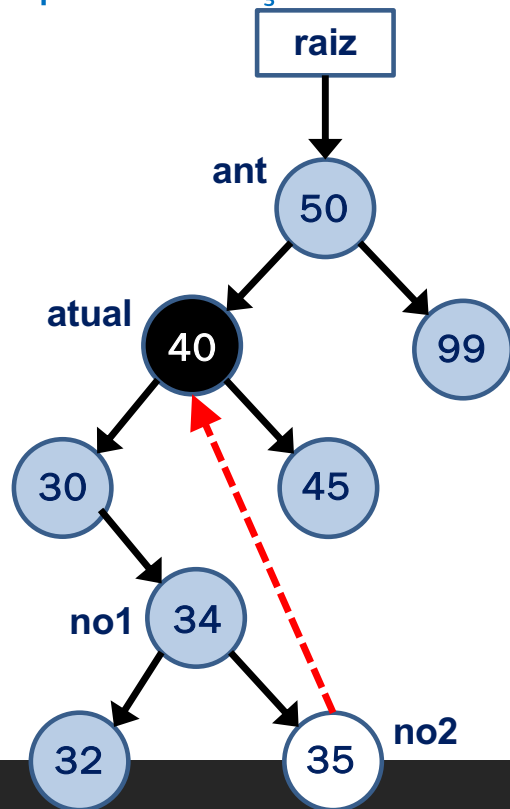
Árvore Binária de Busca: Remoção

Exemplo: remoção de um nó com 1 filho



Árvore Binária de Busca: Remoção

Exemplo: remoção de um nó com 2 filhos



Remoção de um nó com ambos os filhos:

Remover o nó “atual” e substituí-lo pelo nó “mais a direita” da sub-árvore da esquerda do nó “atual”

